Programming in Python
IDC 212
Lecture - 6

September 2, 2025

Contents

1. Exception Handling

Exception Handling

Types of Errors in Programming

In any programming language, two types of errors are possible:

1. **Syntax Errors**
2. **Runtime Errors**

Syntax Errors

The errors which occur because of invalid program structure (grammar) are called **syntax errors**.

Example 1 (missing colon):

```
x = 5
if x > 3
    print("Hello")
```

```
SyntaxError: expected ':'
```

Example 2 (unclosed bracket):

```
nums = [1, 2, 3
print(len(nums))
```

```
SyntaxError: '[' was never closed
```

Note: The programmer must fix all syntax errors before execution can begin.

Runtime Errors (Exceptions)

Also known as **exceptions**. These occur *while* executing the program due to:

- invalid input or unavailable resources,
- logic mistakes,
- operations outside allowed ranges.

Example 1 (IndexError):

```
letters = ["a", "b"]
print(letters[5])
```

```
IndexError: list index out of range
```

Example 2 (ZeroDivisionError):

```
total = 10
n = 0
avg = total / n
```

```
ZeroDivisionError: division by zero
```

What is an Exception?

Exception = an unwanted or unexpected event that occurs *during program execution* and disrupts the normal flow of the program.

Why handle exceptions?

- Prevent sudden termination
- Release/close resources cleanly
- Show helpful error messages or recover if possible

Exceptions Example

- `ZeroDivisionError` — divide by zero
- `NameError` — using a variable that was never defined
- `IndexError` — list index out of range
- `ValueError` — converting an invalid string to a number
- `TypeError` — mixing incompatible types (e.g., int + str)

Uncaught Exception Stops Execution

If an exception is not handled, Python aborts immediately.

Example 1:

```
print("Step 1")
print(10 / 0)          # ZeroDivisionError
print("Step 2")        # never runs
```

Example 2:

```
x = "ten"
y = int(x)             # ValueError
print("Converted:", y) # never runs
```

Key Points

- Exceptions happen *at runtime*; syntax errors happen before running.
- Uncaught exceptions stop the program immediately.
- We catch *specific* exceptions when possible; give clear messages.

Python Exception Hierarchy — Overview

- **BaseException**
  - BaseExceptionGroup
  - GeneratorExit
  - KeyboardInterrupt
  - SystemExit
  - **Exception**

Note: Everything regular code typically catches derives from `Exception`; the other direct children of `BaseException` are special cases.

Under `Exception` (Part 1)

- ArithmeticError
  - FloatingPointError
  - OverflowError
  - ZeroDivisionError
- AssertionError
- AttributeError
- BufferError
- EOFError
- ExceptionGroup [BaseExceptionGroup]
- ImportError
  - ModuleNotFoundError
- LookupError
  - IndexError
  - KeyError
- MemoryError
- NameError
  - UnboundLocalError

Under `Exception` — `OSError` Family

- OSError
  - BlockingIOError
  - ChildProcessError
  - ConnectionError
    - BrokenPipeError
    - ConnectionAbortedError
    - ConnectionRefusedError
    - ConnectionResetError
  - FileExistsError
  - FileNotFoundError
  - InterruptedError
  - IsADirectoryError
  - NotADirectoryError
  - PermissionError
  - ProcessLookupError
  - TimeoutError

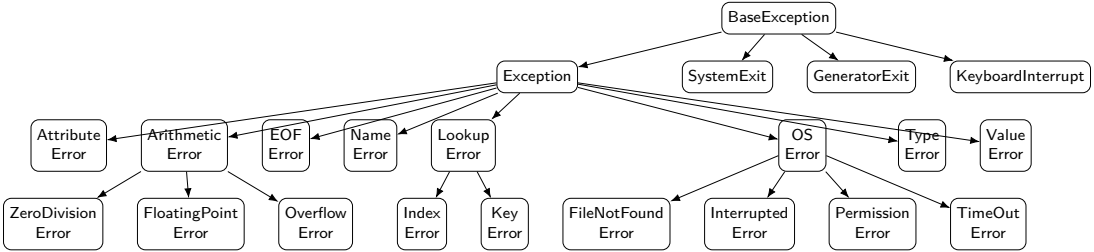
Under `Exception` (Part 2)

- ReferenceError
- RuntimeError
  - NotImplementedError
  - PythonFinalizationError
  - RecursionError
- StopAsyncIteration
- StopIteration
- SyntaxError
  - IndentationError
    - TabError
- SystemError
- TypeError
- ValueError
  - UnicodeError
    - UnicodeDecodeError
    - UnicodeEncodeError
    - UnicodeTranslateError

Warning Hierarchy (under `Exception`)

- Warning
  - BytesWarning
  - DeprecationWarning
  - EncodingWarning
  - FutureWarning
  - ImportWarning
  - PendingDeprecationWarning
  - ResourceWarning
  - RuntimeWarning
  - SyntaxWarning
  - UnicodeWarning
  - UserWarning

Python Exceptions (Hierarchy)



Key Points about Exceptions in Python

- Every **Exception in Python** is defined as a **class**.
- All exception classes are child classes of `BaseException`.
  - Every exception class extends `BaseException` either directly or indirectly.
  - Hence, `BaseException` acts as the root for the Python Exception Hierarchy.
- We mainly focus on `Exception` and its child classes.

Customized Exception Handling using try-except

- It is highly recommended to handle exceptions.
- The code which may raise an exception is called **risky code**.
- We put risky code inside a `try` block, and the corresponding handling code inside an `except` block.

General Form:

```
try:
    # risky code
except XXX:
    # handling code / alternative code
```

Without try-except

Code:

```python
print("stmt-1")
print(10/0)
print("stmt-3")
```

Output:

```
stmt-1
ZeroDivisionError: division by zero
```

With try-except

Code:

```
print("stmt-1")
try:
    print(10/0)
except ZeroDivisionError:
    print(10/2)
print("stmt-3")
```

Output:

```
stmt-1
5.0
stmt-3
```

Example: Handling ValueError

Code:

```python
print("Enter an integer:")
try:
    num = int("abc")   # invalid input
    print("You entered:", num)
except ValueError:
    print("Please enter a valid number")
```

Output:

```
Enter an integer:
Please enter a valid number
```

Example: Handling IndexError

Code:

```
nums = [10, 20, 30]
try:
    print(nums[5])
except IndexError:
    print("Index out of range")
```

Output:

```
Index out of range
```

Example: Handling Multiple Exceptions

Code:

```python
values = ["10", "0", "abc"]

for v in values:
    try:
        print(100 / int(v))
    except ZeroDivisionError:
        print("Division by zero not allowed")
    except ValueError:
        print("Invalid number:", v)
```

Output:

```
10.0
Division by zero not allowed
Invalid number: abc
```

Example: try-except-else

Code:

```python
try:
    num = int("42")
except ValueError:
    print("Invalid input")
else:
    print("Conversion successful:", num)
```

Output:

```
Conversion successful: 42
```

Control Flow in try-except

Template:

```
try:
    step-1
    step-2
    step-3
except SomeError:
    step-4
step-5
```

Scenario A: No error occurs → executes steps 1,2,3,5 → Normal completion

Scenario B: Error occurs at step-2 and is caught by except → executes steps 1,4,5 → Normal completion

Scenario C: Error occurs at step-2 and no matching handler → executes only step-1 → Program stops abnormally

Scenario D: Error happens inside the handler (step-4) or after (step-5) → Abnormal termination.

Observations

1. When an exception arises inside a `try` block, the rest of that block is skipped, even if the error is later handled. $\Rightarrow$ Keep the `try` block minimal, containing only the risky operations.
2. Exceptions are not limited to the `try` part; they can also occur in the `except` or `finally` sections.
3. If code outside of any `try` block raises an exception, the program halts with an error message.

Printing Exception Details in Python

Example:

```python
try:
    result = 10 / 0
except ZeroDivisionError as err:
    print("An exception occurred, details:", err)
```

Output:

```
An exception occurred, details: division by zero
```

Multiple `except` Blocks — Idea

- Different operations may fail in different ways, so we attach a *separate handler* to each expected exception type.
- Python checks handlers **top to bottom** and runs the first one that matches.
- Advantage: clear, targeted messages and easy recovery paths.

Pattern:

- `try:` risky steps
- `except TypeAError:` fix/notify for A
- `except TypeBError:` fix/notify for B

Example 1 — Two Specific Handlers

Code:

```python
a, b = "12", "3"       # try "12","0" or "12","ten"
try:
    a = int(a)
    b = int(b)
    print("Result:", a / b)
except ZeroDivisionError:
    print("Cannot divide by zero.")
except ValueError:
    print("Please enter whole numbers only.")
```

Possible outputs:

```
Result: 4.0
```

```
Cannot divide by zero.
```

```
Please enter whole numbers only.
```

Example 2 — Order of Handlers Matters

Wrong order (general before specific):

```python
x, y = "8", "0"

try:
    x = int(x)
    y = int(y)
    print(x / y)
except ArithmeticError:          # too general for this position
    print("Arithmetic problem") # catches ZeroDivisionError here
except ZeroDivisionError:
    print("Division by zero")    # never reached
```

Output:

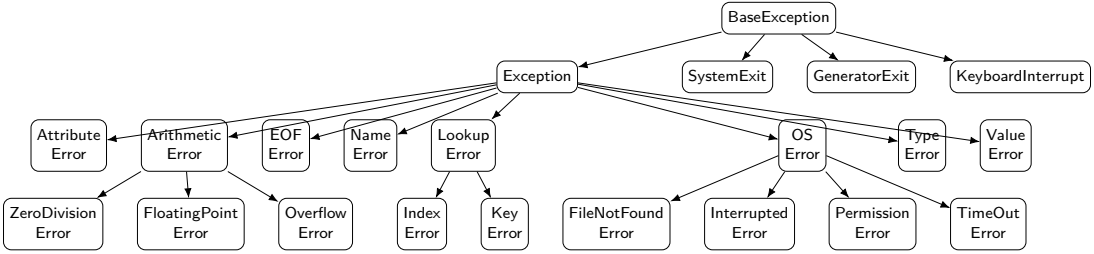
```
Arithmetic problem
```

Example 2 — Order of Handlers Matters

Fix: put specific first

```
try:
    x = int(x_text); y = int(y_text)
    print(x / y)
except ZeroDivisionError:
    print("Division by zero")
except ArithmeticError:
    print("Other arithmetic issue")
```

Python Exceptions (Hierarchy)



One except for Multiple Errors

- A single `except` block can be used to catch more than one type of error.
- List the exception types inside parentheses, separated by commas.
- Optionally, use `as e` to access the actual error message.

Syntax:

```
except (ErrorType1, ErrorType2, ...) as e:
    # handle here
```

Example — Catching Two Kinds of Errors

Code:

```python

first, second = "12", "0"   # try "12","ten" as well

try:
    a = int(first)
    b = int(second)
    print("Quotient:", a / b)
except (ZeroDivisionError, ValueError) as err:
    print("Invalid operation, details:", err)
```

Possible outputs:

```
Quotient: 4.0
```

```
Invalid operation, details: division by zero
```

```
Invalid operation, details: invalid literal for int() with base 10: 'ten'
```

Key Notes

- Parentheses are required when grouping multiple exception types.
- Python treats the group as a tuple of errors.
- Keep the set of exceptions meaningful (don't lump unrelated ones together).
- This style is helpful when different errors can be handled in the same way.

Default except Block

- A **default except block** (written as just `except:`) can catch any kind of exception.
- It is often used to print a general error message when we don't know the exact type of error.
- When combining with other `except` blocks, the default one must always be placed at the **end**.

General Syntax:

```
try:
    # risky operations
except SpecificError:
    # handle this case
except:
    # handle all other errors
```

Example 1 — Specific + Default Handler

Code:

```python
a, b = "15", "0"   # try also: "15","ten"
try:
    a = int(a)
    b = int(b)
    print("Quotient:", a / b)
except ZeroDivisionError:
    print("Cannot divide by zero")
except:
    print("General error: please check your input")
```

Possible Outputs:

```
Quotient: 5.0
```

```
Cannot divide by zero
```

```
General error: please check your input
```

Example 2 — Wrong Order Causes SyntaxError

Code (incorrect):

```python
try:
    print(10 / 0)
except:
    print("Default handler")
except ZeroDivisionError:
    print("Division by zero")
```

Result:

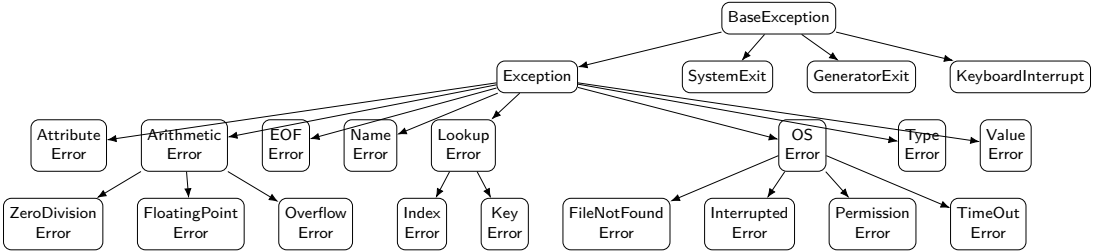
```
SyntaxError: default 'except:' must be last
```

Lesson: Always place the default `except:` block **after all specific handlers**.

Key Points

- Use default `except` for a fallback when no specific error matches.
- Keep specific handlers first; generic one must come last.
- Default handlers are useful for showing user-friendly messages.
- Don't rely only on default `except`; prefer targeted handlers when possible.

Python Exceptions (Hierarchy)



Different Forms of except Blocks

Note: Python allows several variations when writing an `except` block.

1. Catch a specific error (no variable):

```
except ZeroDivisionError:
    # handle division by zero
```

2. Catch a specific error and bind it to a variable:

```
except ZeroDivisionError as err:
    print("Problem is:", err)
```

3. Catch more than one error type:

```
except (ZeroDivisionError, ValueError):
    # handle both situations
```

4. Catch multiple error types and bind to variable:

```
except (ZeroDivisionError, ValueError) as err:
    print("Problem is:", err)
```

The `finally` Block

- Cleanup code (like closing files or releasing resources) should not be kept inside `try` or `except` alone.
- Why? Because:
  - If an exception occurs inside `try`, later lines there won't run.
  - If no exception occurs, the `except` part won't run at all.
- The `finally` block is the right place for cleanup tasks, because it is **guaranteed to execute**:
  - Exception raised or not.
  - Exception handled or not.

General Form of `finally`

```python
try:
    # risky operations
except SomeError:
    # handle the problem
finally:
    # cleanup code (runs no matter what)
```

Key Point: The `finally` block always executes before the program exits from the `try/except` structure.

Case 1: No Exception

Code:

```python
try:
    print("In try block")
except:
    print("In except block")
finally:
    print("In finally block")
```

Output:

```
In try block
In finally block
```

Case 2: Exception Raised and Handled

Code:

```python
try:
    print("In try block")
    print(10 / 0)
except ZeroDivisionError:
    print("In except block")
finally:
    print("In finally block")
```

Output:

```
In try block
In except block
In finally block
```

Case 3: Exception Raised but Not Handled

Code:

```python
try:
    print("In try block")
    print(10 / 0)
except NameError:
    print("In except block")
finally:
    print("In finally block")
```

Output:

```
In try block
In finally block
ZeroDivisionError: division by zero
```

Summary

- Use `finally` to ensure important cleanup tasks (closing files, releasing locks, freeing resources).
- It runs:
  - When no exception occurs.
  - When an exception occurs and is handled.
  - When an exception occurs but is not handled.
- **Best Practice:** Keep `finally` blocks short and limited to essential cleanup.

MCQ 1: No Exception Case

What will be the output?

```python
try:
    print("A")
    print("B")
except ZeroDivisionError:
    print("C")
finally:
    print("D")
    print("E")
```

1. A, B, C
2. A, B, D, E
3. A, D, E
4. Only D, E

Answer – MCQ 1

Correct Answer: (2) A, B, D, E

- No exception is raised, so the `except` block is skipped.
- The `finally` block always executes, printing D and E.

MCQ 2: Exception Handled

What will be the output?

```python
try:
    print("X")
    print(10/0)
except ZeroDivisionError:
    print("Y")
finally:
    print("Z")
```

1. X, Y
2. X, Z
3. X, Y, Z
4. Only Z

Answer – MCQ 2

Correct Answer: (3) X, Y, Z

- Division by zero occurs and is caught by `except` (prints Y).
- The `finally` block then runs (prints Z).

MCQ 3: Exception Not Handled

What will be the output?

```python
try:
    print("One")
    print(10/0)
except NameError:
    print("Two")
finally:
    print("Three")
```

1. One, Two, Three
2. One, Three, ZeroDivisionError
3. One, ZeroDivisionError
4. One, Two

Answer – MCQ 3

Correct Answer: (2) One, Three, ZeroDivisionError

- The handler is for `NameError`, but the error is `ZeroDivisionError`.
- `finally` still runs (prints Three), then the unhandled error terminates the program.

MCQ 4: Exception Inside except Block

What will be the output?

```python
try:
    print("Hello")
    print(10/0)
except ZeroDivisionError:
    print(10/"a")   # raises TypeError
finally:
    print("End")
```

1. Hello, End
2. Hello, TypeError
3. Hello, End, TypeError
4. Only End

Answer – MCQ 4

Correct Answer: (3) Hello, End, TypeError

- The zero-division transfers control to `except`.
- Inside `except`, another exception (`TypeError`) is raised.
- Before it is reported, `finally` runs (prints End).

MCQ 5: Exception in finally Block

What will be the output?

```python
try:
    print("Start")
except:
    print("Except")
finally:
    print(10/0)
```

1. Start
2. Start, Except
3. Start, ZeroDivisionError
4. Except, ZeroDivisionError

Answer – MCQ 5

Correct Answer: (3) Start, ZeroDivisionError

- `try` prints Start; `except` is skipped.
- `finally` raises a new error, which ends the program.

Lab Problem 1: Velocity with Exception Handling

Compute the velocity $v = \frac{d}{t}$ where:

- `d` is distance and `t` is time (both provided by the user).
- Handle two error cases:
  - **ZeroDivisionError** when `t = 0`.
  - **ValueError** when the input is not numeric.
- Always print `"Physics calculation finished"` using a `finally` block.

Solution: try/except/finally

Python code:

```python
try:
    d_text = input("Enter distance (meters): ")
    t_text = input("Enter time (seconds): ")

    d = float(d_text)            # may raise ValueError
    t = float(t_text)            # may raise ValueError

    v = d / t                    # may raise ZeroDivisionError
    print("Velocity:", v, "m/s")

except ZeroDivisionError:
    print("Time cannot be zero. Please measure time > 0.")
except ValueError:
    print("Inputs must be numeric (e.g., 12.5).")
finally:
    print("Physics experiment finished")
```

Solution: Sample Runs

Case A: valid inputs

```
Enter distance (meters): 10
Enter time (seconds): 2
Velocity: 5.0 m/s
Physics experiment finished
```

Case B: division by zero

```
Enter distance (meters): 10
Enter time (seconds): 0
Time cannot be zero. Please measure time > 0.
Physics experiment finished
```

Case C: non-numeric input

```
Enter distance (meters): eight
Enter time (seconds): 2
Inputs must be numeric (e.g., 12.5).
Physics experiment finished
```

Optional Variant: One except for Two Errors

Alternative style:

```python
try:
    d = float(input("Distance (m): "))
    t = float(input("Time (s): "))
    print("Velocity:", d / t, "m/s")
except (ValueError, ZeroDivisionError) as err:
    print("Cannot compute velocity:", err)
finally:
    print("Physics experiment finished")
```

Lab Problem 2: DNA Sequence Validation

A program takes a DNA sequence (string of `A, T, G, C`).

- If the input contains invalid characters, raise and catch a `ValueError`.
- If the input is valid, print `"Sequence accepted"`.
- Always print `"DNA processing finished"` using a `finally` block.

Solution: Using try/except/finally

Python code:

```python
try:
    seq = input("Enter DNA sequence: ").upper()
    valid = set("ATGC")

    # check validity
    if not set(seq).issubset(valid):
        raise ValueError("Invalid DNA sequence")

    print("Sequence accepted:", seq)

except ValueError as err:
    print("Error:", err)
finally:
    print("DNA processing finished")
```

Solution: Sample Runs

Case A: Valid sequence

```
Enter DNA sequence: ATGCCGTA
Sequence accepted: ATGCCGTA
DNA processing finished
```

Case B: Invalid sequence

```
Enter DNA sequence: ATBXZ
Error: Invalid DNA sequence
DNA processing finished
```

Thank You!